Phuthiphong Wongchantib 660610840

CounterClockwise-Only Steering Robot — Detour Steer Algorithm

Drive on, steer less, come back

Robot. One DC motor: turning it one way *steers* the wheel module, *counter-clockwise only* (one-way clutch); turning it the other way *drives* forward. It can never steer and drive at the same time. The old method steers straight at the goal, so a goal a little to the *right* (φ close to 360°) costs almost a full turn. **Idea.** Two straight legs whose headings differ by more than 180° can reach *any* direction, so the robot may instead drive a straight on, steer only β^* ($180^\circ < \beta^* < \varphi$), and drive b to the goal. The best β^* has a closed form. Headings are counter-clockwise from the $+x$ axis. ω steer speed, v drive speed, t_s settle time, t_b stop time (drive → steer), tol tolerance, r_{goal} arrival radius. **On the robot** (`mor_luam`, ESP32) the module turns counter-clockwise seen from above: the same frame as this page. The firmware helper name cwErrorDeg is historical; its sign is counter-clockwise. Algorithms 1.1–1.5 plan; 1.6–1.7 are how the robot steers and drives each leg.

Algorithm 1.1: Counter-Clockwise Heading Helpers

Procedure : *Bearing*(Δx, Δy) // CounterClockwise heading that points along $(\Delta x, \Delta y)$
  return $\operatorname{atan2}(\Delta y, \Delta x) \bmod 2\pi$;
end

Procedure : *SteeringError*(θ, h) // How far the wheel must turn counter-clockwise: $0 \ldots 2\pi$
  return $(\theta - h) \bmod 2\pi$;
end

Procedure : *InTolerance*(e) // Close enough on either side of the target
  return $e \le tol$ **or** $e \ge 2\pi - tol$; // Just *past* the target reads as $e \approx 2\pi$
end

Algorithm 1.2: Optimal Detour Angle in Closed Form, $O(1)$

Procedure : *OptimalDetourAngle*(φ, d) // φ: counter-clockwise angle to the goal, d: distance
  if $\varphi \le \pi$ **then return None**; // Theorem 1: steering straight is optimal
  $k \leftarrow (d\,\omega / v)\,|\sin\varphi|$; // One number decides everything
  if $k \ge 2$ **then return None**; // Theorem 3: time keeps falling up to $\beta = \varphi$
  $\beta^* \leftarrow 2\pi - \arccos(k - 1)$; // The only root of $dT/d\beta = 0$ in $(\pi, 2\pi)$
  if $\beta^* \ge \varphi$ **then return None**; // The best angle is not a detour
  return β^*;
end

Algorithm 1.3: Plan One Leg: Direct or Detour, $O(1)$

Procedure : *PlanLeg*(φ, d) // Returns (kind, a, β, b, time)
  $T_D \leftarrow \varphi/\omega + t_s + d/v$; // Old method: steer φ, drive d
  $\beta^* \leftarrow$ *OptimalDetourAngle*(φ, d);
  if $\beta^* =$ **None then return** (DIRECT, 0, φ, d, T_D);
  $a \leftarrow d \sin(\beta^* - \varphi) / \sin\beta^*$; // Leg 1: heading unchanged, no steer
  $b \leftarrow d \sin\varphi / \sin\beta^*$; // Leg 2: heading β^*, ends at the goal
  $T_R \leftarrow a/v + t_b + \beta^*/\omega + t_s + b/v$; // One extra stop between the legs
  if $T_R < T_D$ **then return** (DETOUR, a, β^*, b, T_R); // Use the detour only if it is really faster
  return (DIRECT, 0, φ, d, T_D);
end

Algorithm 1.4: Closed-Loop Detour Steer to One Goal (plan again after every move)

```
Procedure : DetourSteer(p, h, g)                          // p: position, h: wheel heading, g: goal
    while ‖g − p‖ > r_goal do
        d ← ‖g − p‖;    e ← SteeringError(Bearing(g − p), h);
        if InTolerance(e) then                            // Already aimed: drive to the goal
            DriveReaim(d);                                // Algorithm 1.7
        else
            plan ← PlanLeg(e, d);
            if plan.kind = DETOUR then
                DriveReaim(plan.a);                       // Leg 1: straight on, no steer
                Stop();                                   // t_b: switch the motor from drive to steer
            else
                SteerOneWay(Bearing(g − p));              // Algorithm 1.6: stops a little short on purpose
            end
        end
        p ← MeasurePosition();    h ← MeasureHeading();   // AS5600 wheel angle + IMU body heading; T4
        // Short by ε: e = ε, a small extra steer. Past by ε: e = 2π − ε, PlanLeg detours
    end
    return (p, h);
end
```

Algorithm 1.5: End-to-End Route Following with Detour Steer

```
Procedure : FollowRouteDetour(W)                          // Master procedure: visit the goals W in order
    p ← (0, 0);    h ← 0;                                 // Start pose
    foreach g ∈ W do
        (p, h) ← DetourSteer(p, h, g);
    end
    Display total time;          // Goal 1 m at 350°: a=1.03 m, β*=254°, b=0.18 m, 9.34 s (old method: 9.88 s)
end
```

Algorithm 1.6: Steer One Way: Stop a Little Short on Purpose (robot, every 10 ms)

```
Procedure : SteerOneWay(θ)                                // θ: the heading to reach
    while true do                                         // Every control tick, 10 ms
        h ← MeasureHeading();          // Wheel (AS5600) + body (IMU): the body twists as it steers
        e ← SteeringError(θ, h);    r ← steering speed over 50 ms;
        if e < π and e ≤ e_land + r c then break;         // Cut early: it coasts on to ≈ e_land short
        if InTolerance(e) and (e > π or r < r_min) then break; // Past, or standing: a push from rest jumps past
        steering motor on;
    end
    steering motor off; wait until the wheel stops and the body turns < 5°/s (at most 0.4 s);
    c ← (1 − λ) c + λ (angle coasted / r_cut);            // Learn the coast time c
end
```

Algorithm 1.7: Drive with Early Re-aim (one motor: it cannot steer while it drives)

```
1  Procedure : DriveReaim(ℓ)                          // ℓ: leg length on the current heading h
2      q ← p + ℓ (cos h, sin h);                      // The end of this leg
3      while true do                                  // Every control tick, 10 ms
4          (f, s) ← q − p seen from the wheel;        // f ahead, s beside (+: steering side)
5          if ‖q − p‖ ≤ r_goal or f ≤ 0 then break;   // Arrived, or level with it
6          if s > r_goal and f ≤ max(m, R s) then break;   // Re-aim is then only arctan(1/R)
7          drive motor on;
8      end
9      drive motor off;    p ← MeasurePosition();     // Algorithm 1.4 plans again from here
10 end
```

Correctness, complexity, results and closest prior work

For a goal at counter-clockwise angle $\varphi > \pi$ and distance d, a detour with final heading β takes

$$T(\beta) = \frac{\beta}{\omega} + t_s + t_b + \frac{d}{v}\, g(\beta), \qquad g(\beta) = \frac{\sin(\beta - \varphi) + \sin\varphi}{\sin\beta}, \qquad g'(\beta) = \frac{\sin\varphi}{1 + \cos\beta}.$$

T1. If $\varphi \le 180°$, steering straight at the goal is optimal. Two headings less than 180° apart only reach directions between them, so every path steers at least φ; every path also drives at least d. The direct path does both.

T2. If $\varphi > 180°$, a best path has at most 2 legs, with headings 0 and β. The shortest way to write the goal vector with headings in $[0, \beta]$ uses the two headings on either side of φ (the optimum of a 2-row linear program has 2 non-zero variables).

T3. On $(\pi, 2\pi)$, $T'(\beta) = 1/\omega - (d/v)|\sin\varphi|/(1 + \cos\beta)$ is strictly increasing, so T has exactly one minimum: $\cos\beta^* = k - 1$ with $k = (d\omega/v)|\sin\varphi|$. If $k \ge 2$, T keeps falling up to $\beta = \varphi$, which is the direct path.

T4. After leg 1, *PlanLeg(β^*, b)* returns "steer β^*, drive b". Any other detour from there is a longer leg 1 plus one more stop t_b, which T3 already beat. So the planner can run again after every move (closed loop) without changing its mind.

O. *PlanLeg()* is $O(1)$ in time and memory: 0.54 µs per call in Python, against 505 µs for a 3600-point search over β (944×). T1–T4 are also checked by brute force in `detour_steer.py`: closed form vs a 20 000-point grid, vs every 2-leg path on a 2° grid, and 2000 replans. All PASS.

Unknown stop error. Nobody knows in advance whether the wheel stops a little past or a little short. The robot reads the real wheel angle (AS5600) and body heading (IMU) after it stops, and *DetourSteer()* plans again from there; *PlanLeg()* never uses the error. Inside $\pm tol$ the robot drives on its real heading, and a miss of more than r_{goal} is planned again.

Why stop short on purpose (Algorithm 1.6). Past its angle by more than *tol*, a one-way wheel needs almost a full turn; short of it, only a little more. So the robot cuts the motor early and lets the wheel coast to $e_{land} = 2°$ short of the angle (c, the coast time, is learned on every steer). A wheel standing inside the band is never pushed again: from rest the motor must build up power and the wheel jumps past (measured: 9°). Stopping short also means a miss always leaves the goal on the *steering side* of the line the robot drives on, so *DriveReaim()* fixes it with a small turn ($\arctan(1/R) = 18°$ for $R = 3$, $m = 2$ cm) instead of a 90° turn after passing the goal.

Results (5000 random goals, $d \in [0.3, 2]$ m, $v = 0.25$ m/s, $t_b = 0.2$ s, $\omega = 60°/$s; the same goals and stop errors for both; mean time per goal):

wheel stops	bias	sd	old method (s)	detour (s)	saved
exactly	0°	0°	7.56	7.53	0.4 %
either way, random	0°	2°	8.42	8.23	2.3 %
either way, random	0°	3°	9.43	8.78	6.9 %
mostly short	−2°	2°	8.08	8.03	0.7 %
mostly past	+2°	2°	10.74	9.36	12.8 %
mostly past	+2°	3°	11.87	9.70	18.3 %

In every case Detour Steer is slower on 0 % of the goals. With exact stops the gain is small on average but up to 28 % for a goal just to the right. The more the wheel stops past its target, the more it gains: an overshoot no longer costs a full extra turn. Slower steering gives more gain (at 30°/s, sd 2°: 4.8 %).

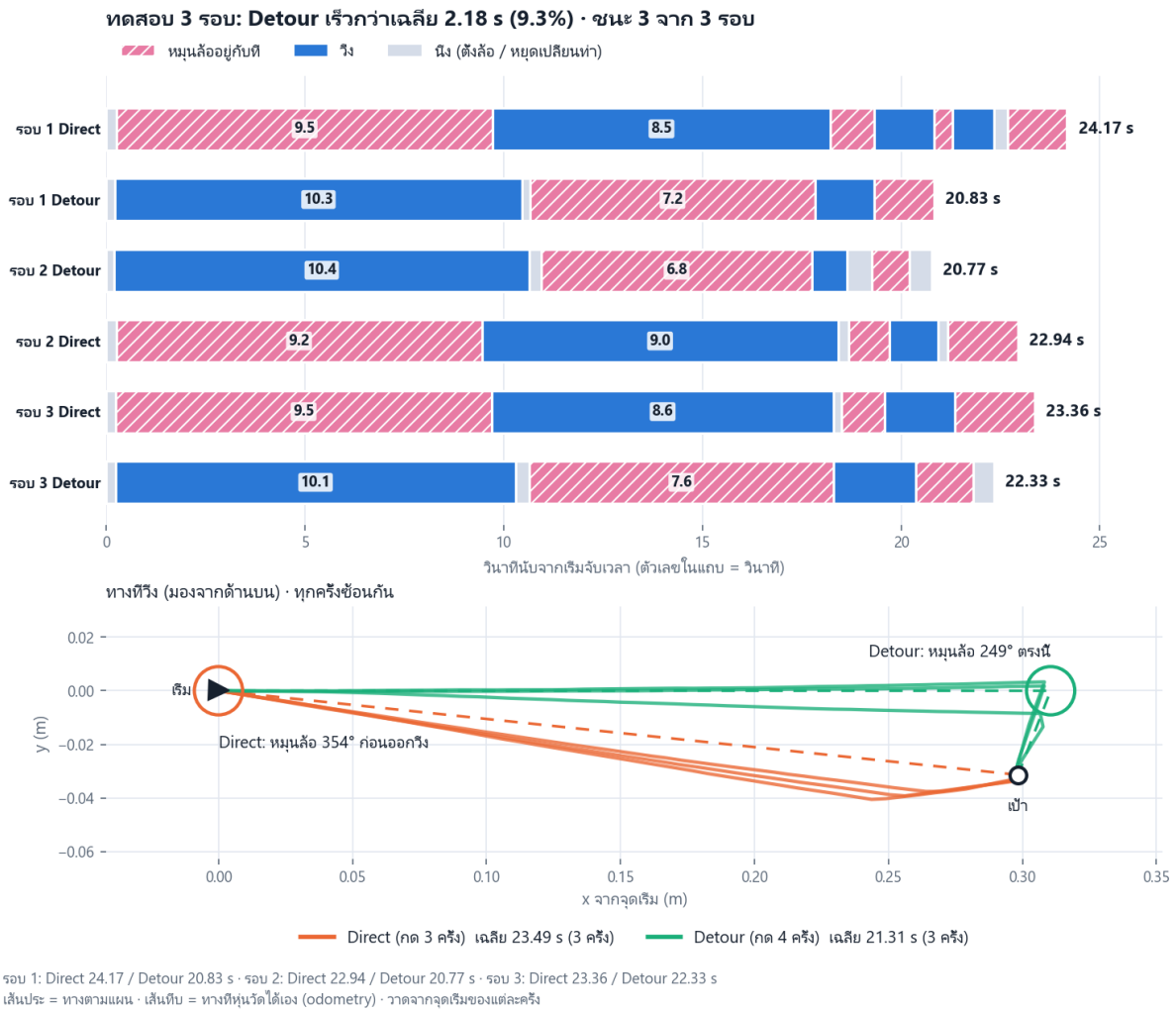
Results on the robot (`mor_luam`: $v = 0.03$ m/s, $\omega = 35°/$s its measured steering speed, $t_b = 0.2$ s, $r_{goal} = 2.5$ mm). The same task for both: before the clock starts the wheel is turned to the start heading and is still; the goal is then placed $d = 0.30$ m away at $\varphi = 354°$ from where the wheel really points. The robot times each run itself and drives back to the start (not timed). 3 rounds, order old–detour, detour–old, old–detour:

round	old method (s)	Detour Steer (s)	saved (s)
1	24.17	20.83	3.34
2	22.94	20.77	2.17
3	23.36	22.33	1.03
mean	23.49	21.31	2.18 (9.3 %)

Detour Steer was faster in 3 of 3 rounds. It steered 3.8 s less (8.5 vs 12.4 s) and drove 1.2 s more (11.8 vs 10.6 s). The plan predicted 18.8 vs 20.2 s; the floor gave a bigger gap, because the old method's near-full turn also needs more correction at the end. All 6 runs ended within 2.5 mm of the goal.

Closest prior work, and the difference. One-motor steer-and-drive mechanisms (US 8,424,627 B2, 2013; Sekine & Mizuuchi, ICINCO 2022) give no planner. Dubins cars that can turn one way only (Bakolas & Tsiotras, JOTA 2011; Patsko & Fedotov, 2018–2019) turn *while moving*. Turn-cost routing (angular-metric and quadratic TSP) orders waypoints, and predictive coast stopping (US 8,571,394 B2) stops early. None of the work found gives this closed-form detour for a robot that must *stop to steer, counter-clockwise only*. "Not found" is not proof that nobody did it.

Figure 1. The 3 rounds on the robot, from its own record



Top: what each run did over time (hatched pink = steering in place, blue = driving, grey = still). The old method first steers ≈ 9.4 s in place, then drives; Detour Steer drives at once and steers less later. **Bottom:** the paths the robot measured (odometry), all six from one start, seen from above; dashed = the plan. (The robot steers counter-clockwise, so here the goal is to the right.)

Current robot profile and two-metre comparison (27 October 2026)

The numerical benchmark above uses a theoretical reference speed of 0.25 m/s; it is not the physical robot's speed. The measured planning profile is $v = 0.03$ m/s, $\omega = 35°/s$, and goal radius 0.0025 m. For the longer comparison, split a total outbound length of 2 m into seven equal legs, each 2/7 m, bending 6° clockwise from the preceding bearing. Both planners visit the same waypoints; physical steering remains counter-clockwise. The ideal model gives Direct 137.82 s / 2.00 m and Detour 110.52 s / 2.17 m, with four detours. These are model predictions, not new real-robot measurements. The return to start (about 1.96 m) is additional and untimed. Three paired rounds contain six runs. No autonomous motion follows from saving settings.